Task 17: Inter-service communication

Aim:

To enable inter-service communication in a microservices architecture using REST APIs, where one microservice consumes data from another and handles responses and failures gracefully.

Algorithm:

1. Create **Service B (Provider)** using Express.js to expose REST endpoints (/data, /error).
2. Start Service B on a specific port (e.g., 4000).
3. Create **Service A (Consumer)** using Express.js.
4. In Service A, use axios to send HTTP requests to Service B.
5. Implement an endpoint (/fetch-data) in Service A to call Service B's /data API.
6. Use **async/await with try-catch** to handle API responses.
7. If the request is successful, return the response from Service B to the client.
8. If an error occurs (timeout, server error, or service down), handle it gracefully.
9. Provide a **fallback response** in case of failure.
10. Test all endpoints using Postman or browser.

Code:

Server.js

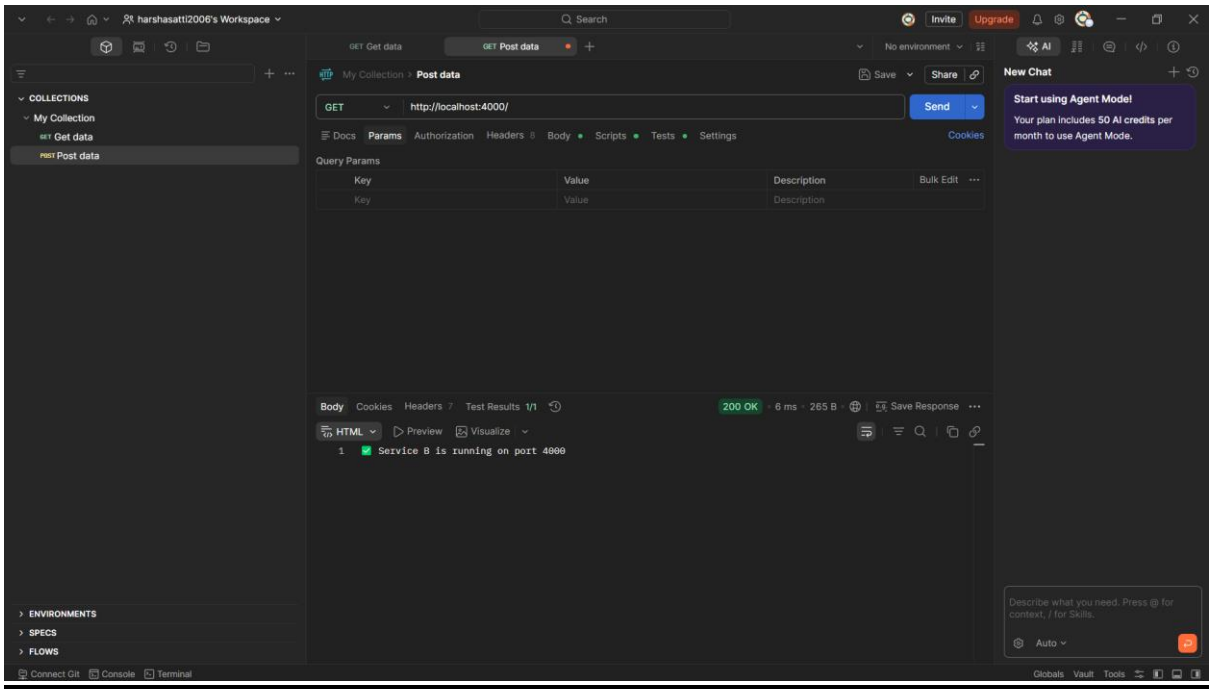
```
const express = require("express");

const axios = require("axios");


/* ==========================================

  SERVICE B (Provider)

  ========================================== */

const serviceB = express();


// Root route (fix for Cannot GET /)

serviceB.get("/", (req, res) => {

   res.send("✅ Service B is running on port 4000");

});


// Normal API
```

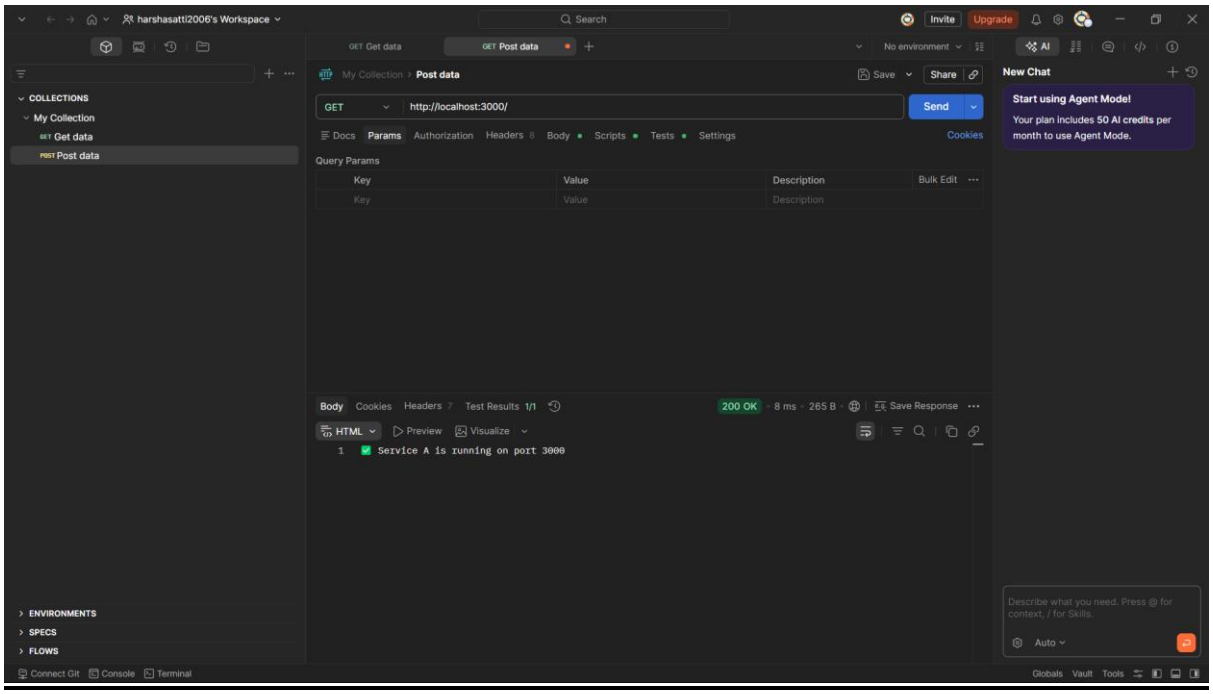
```
serviceB.get("/data", (req, res) => {
    res.json({
        success: true,
        message: "Data from Service B",
        data: {
            id: 101,
            name: "Microservice Example"
        }
    });
});

// Error simulation
serviceB.get("/error", (req, res) => {
    res.status(500).json({
        success: false,
        message: "Service B Internal Error"
    });
});

serviceB.listen(4000, () => {
    console.log("🚀 Service B running at http://localhost:4000");
});

/* ==========================================
   SERVICE A (Consumer)
   ========================================== */
const app = express();

// Root route (fix for Cannot GET /)
```

```
app.get("/", (req, res) => {
    res.send("✅ Service A is running on port 3000");
});

// Call Service B (SUCCESS)
app.get("/fetch-data", async (req, res) => {
    try {
        const response = await axios.get("http://localhost:4000/data", {
            timeout: 3000
        });

        res.json({
            success: true,
            service: "A",
            dataFromB: response.data
        });

    } catch (error) {
        res.status(500).json({
            success: false,
            message: "Error communicating with Service B",
            error: error.message,
            fallback: {
                data: "Default fallback data"
            }
        });
    }
});
```

```
// Call Service B (ERROR HANDLING)
app.get("/fetch-error", async (req, res) => {
    try {
        const response = await axios.get("http://localhost:4000/error");

        res.json(response.data);

    } catch (error) {
        res.status(500).json({
            success: false,
            message: "Handled Service B failure",
            error: error.response ? error.response.data : error.message,
            fallback: "Using backup response"
        });
    }
});

app.listen(3000, () => {
    console.log("🚀 Service A running at http://localhost:3000");
});
```

Outputs:





Result: Successfully implemented REST-based inter-service communication between two microservices. The consumer service (Service A) was able to fetch data from the provider service (Service B) and handle errors such as server failure and connection issues gracefully using fallback responses.